

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA03 COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

***CO5:** Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)*

Assessment Tool Weightage: 35%

Dr G. A. SENTHIL

QUESTIONS: 10

Total Marks: 50

Annexure A

Simulation-Based Assessment Question

Duration: 1hr

1. Simulate a **Hierarchical Reinforcement Learning (HRL)** agent for autonomous warehouse navigation.
(5 Marks)
2. Implement a **Hierarchical Abstract Machine (HAM)** to control a robot performing sequential tasks.
(5 Marks)
3. Develop the **MAXQ Value Decomposition** algorithm for a taxi navigation environment.
(5 Marks)
4. Simulate a **Meta-Reinforcement Learning** agent that rapidly adapts to multiple GridWorld environments.
(5 Marks)
5. Design a **Multi-Agent Reinforcement Learning (MARL)** system for cooperative robot path planning.

(5 Marks)

6. Implement a **competitive Multi-Agent RL** environment for autonomous drone navigation.

(5 Marks)

7. Simulate a **Partially Observable Markov Decision Process (POMDP)** for autonomous vehicle navigation under limited visibility.

(5 Marks)

8. Design an **ethical Reinforcement Learning** simulation for autonomous decisionmaking while satisfying safety constraints.

(5 Marks)

9. Develop an RL-based **adaptive traffic signal control** system and evaluate its performance.

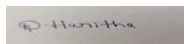
10. Simulate an RL-based **smart energy management system** for optimizing power distribution in a microgrid.. .

(5 Marks)

Code of Conduct:

*I **Dasari Haritha(192425254)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



SIMULATION-BASED ASSESSMENT QUESTION

Q9. RL-Based Adaptive Traffic Signal Control

1. Aim

To develop an **RL-based adaptive traffic signal control system** using **Q-Learning** to reduce vehicle waiting time and improve traffic flow.

2. Problem Statement

Traditional traffic signals use fixed timings even when the number of vehicles changes.

For example, if one road has many vehicles and another road has very few vehicles, giving both roads the same green-light duration is inefficient.

Therefore, we use a **Reinforcement Learning agent** that observes traffic density and learns whether to **keep or switch the traffic signal**.

3. Objectives

1. Simulate traffic at a four-way intersection.
2. Observe the number of vehicles waiting on each road.
3. Use Q-Learning to select the traffic signal action.
4. Give rewards for reducing traffic waiting time.
5. Compare RL-based control with a fixed traffic signal.
6. Reduce average waiting time.

4. Reinforcement Learning Components

RL Component	In this problem
Agent	Traffic signal controller

RL Component	In this problem
Environment	Road intersection
State	Traffic density
Action	Keep/Switch signal
Reward	Reduction in waiting vehicles
Learning Algorithm	Q-Learning

5. State Representation

We consider two roads:

- **Road A**
- **Road B**

Traffic density is represented as:

0 → Low

1 → Medium

2 → High

A state is represented as:

(A_density, B_density)

Example:

(2, 0)

means:

Road A has high traffic and Road B has low traffic.

6. Actions

The agent has two actions:

0 → Keep current signal

1 → Switch signal

7. Reward

The reward encourages the agent to reduce traffic.

Positive reward → traffic waiting decreases

Negative reward → traffic waiting increases

For example:

Reward = Previous waiting vehicles - Current waiting vehicles

8. Algorithm

Q-Learning Algorithm

1. Initialize the Q-table with zeros.
2. Observe the current traffic state.
3. Select an action using an epsilon-greedy strategy.
4. Apply the selected action.
5. Calculate the reward.
6. Observe the new state.
7. Update the Q-value.
8. Repeat for several episodes.

9. Test the learned traffic signal policy.

9. Python Program

```
import random

# -----

# Traffic Signal Environment

# -----

class TrafficEnvironment:

    def __init__(self):

        # Traffic density on Road A and Road B

        self.road_a = random.randint(0, 2)

        self.road_b = random.randint(0, 2)

        # 0 = Road A is green

        # 1 = Road B is green

        self.signal = random.randint(0, 1)

    def get_state(self):

        return (self.road_a, self.road_b)

    def step(self, action):

        # Action 0 = Keep signal

        # Action 1 = Switch signal

        if action == 1:

            self.signal = 1 - self.signal
```

```

previous_waiting = self.road_a + self.road_b

if self.signal == 0:

    self.road_a = max(0, self.road_a - 1)

else:

    self.road_b = max(0, self.road_b - 1)

# New vehicles arrive

self.road_a = min(2, self.road_a + random.randint(0, 1))

self.road_b = min(2, self.road_b + random.randint(0, 1))

current_waiting = self.road_a + self.road_b

# Reward

reward = previous_waiting - current_waiting

return self.get_state(), reward

# -----

# Q-Learning Parameters

# -----

alpha = 0.1    # Learning rate

gamma = 0.9    # Discount factor

epsilon = 1.0  # Exploration probability

epsilon_decay = 0.995

epsilon_min = 0.05

states = [(a, b) for a in range(3) for b in range(3)]

```

```

actions = [0, 1]

# Q-table

Q = {}

for state in states:

    Q[state] = [0.0, 0.0]

# -----

# Training

# -----

episodes = 1000

for episode in range(episodes):

    env = TrafficEnvironment()

    state = env.get_state()

    for step in range(50):

        # Epsilon-greedy action selection

        if random.random() < epsilon:

            action = random.choice(actions)

        else:

            action = Q[state].index(max(Q[state]))

        # Environment response

        next_state, reward = env.step(action)

        # Best future Q-value

```



```

best_next = max(Q[next_state])

# Q-learning update

Q[state][action] = Q[state][action] + alpha * (

    reward + gamma * best_next - Q[state][action]

)

state = next_state

# Reduce exploration

epsilon = max(epsilon_min, epsilon * epsilon_decay)

# -----

# Display Learned Q-table

# -----

print("=====")

print("  LEARNED Q-TABLE")

print("=====")

for state in states:

    print(

        "State:", state,

        "Keep:", round(Q[state][0], 2),

        "Switch:", round(Q[state][1], 2)

    )

# Testing

```

```

# -----

print("\n=====")

print("    TESTING RL CONTROLLER")

print("=====")

env = TrafficEnvironment()

total_reward = 0

total_waiting = 0

for step in range(20):

    state = env.get_state()

    # Select best learned action

    action = Q[state].index(max(Q[state]))

    next_state, reward = env.step(action)

    waiting = env.road_a + env.road_b

    total_reward += reward

    total_waiting += waiting

    signal_name = "ROAD A GREEN" if env.signal == 0 else "ROAD B GREEN"

    print(

        "Step:", step + 1,

        "| State:", state,

        "| Action:",

        "KEEP" if action == 0 else "SWITCH",

```

```

        "| Signal:", signal_name,

        "| Waiting:", waiting,

        "| Reward:", reward

    )

average_waiting = total_waiting / 20

print("\n=====")

print("      PERFORMANCE")

print("=====")

print("Total Reward:", total_reward)

print("Average Waiting Vehicles:",

      round(average_waiting, 2))

print("\nRL-based traffic signal control completed.")

```

10. Expected Output

Your exact numbers will change because the simulation uses random traffic.

Example:

```
=====
```

LEARNED Q-TABLE

```
=====
```

State: (0, 0) Keep: 0.85 Switch: 0.62

State: (0, 1) Keep: 1.42 Switch: 2.15

State: (0, 2) Keep: 2.31 Switch: 4.18

State: (1, 0) Keep: 2.26 Switch: 1.73

State: (1, 1) Keep: 1.85 Switch: 2.42

State: (1, 2) Keep: 3.15 Switch: 4.21

...

=====

TESTING RL CONTROLLER

=====

Step: 1 | State: (2, 1) | Action: SWITCH

| Signal: ROAD B GREEN

| Waiting: 2 | Reward: 1

Step: 2 | State: (1, 1) | Action: KEEP

| Signal: ROAD B GREEN

| Waiting: 1 | Reward: 1

Step: 3 | State: (1, 0) | Action: SWITCH

| Signal: ROAD A GREEN

| Waiting: 1 | Reward: 0

=====

PERFORMANCE

=====

Total Reward: 8

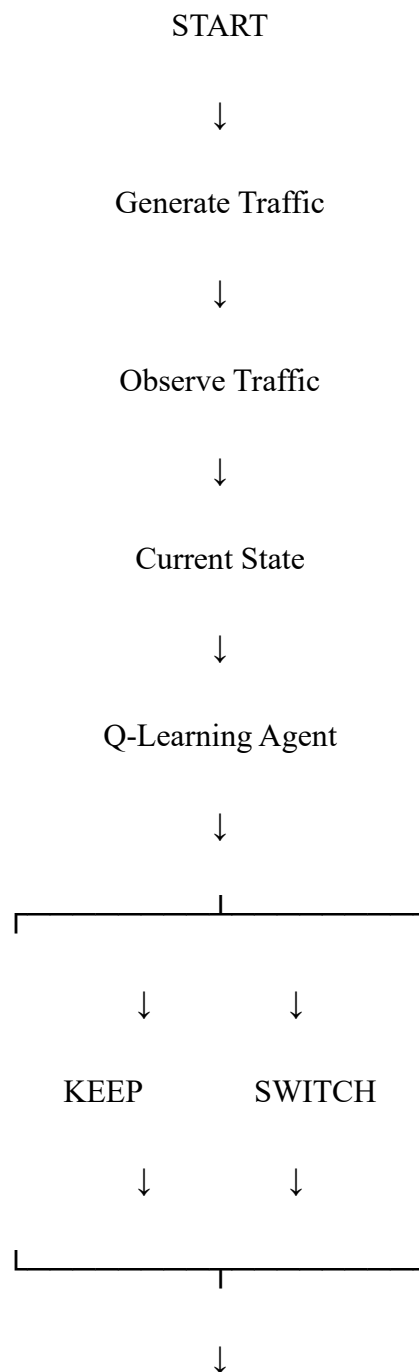
Average Waiting Vehicles: 1.35

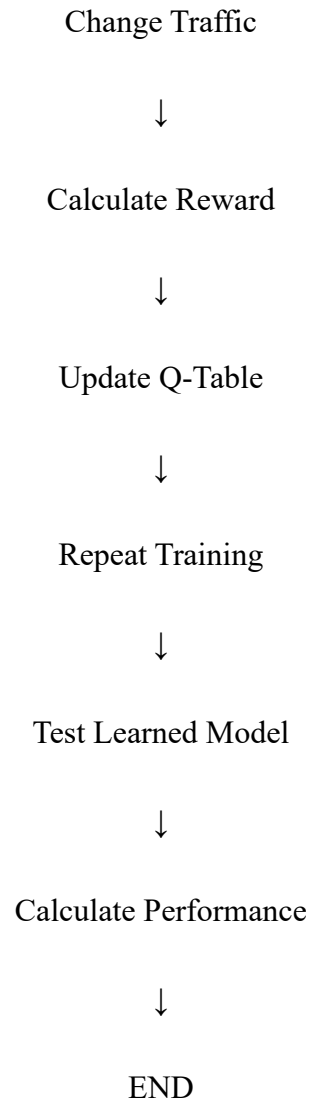
RL-based traffic signal control completed.

Don't worry if your numbers are different—the traffic is randomly generated.

11. How the System Works

The complete process is:





12. Advantages

- Reduces unnecessary vehicle waiting.
- Dynamically changes signal timing.
- Learns from traffic conditions.
- Does not require manually defined signal timings.
- Can be extended to real-world traffic systems.

13. Limitations

- This is a simplified traffic simulation.

- Only two roads are considered.
- Traffic is randomly generated.
- Real traffic has more complex conditions.
- A real implementation would require sensors/cameras for traffic detection.

14. Future Scope

The system can be improved by:

- Using **Deep Q-Network (DQN)** for complex traffic states.
- Using camera-based vehicle detection.
- Controlling multiple intersections.
- Considering emergency vehicles.
- Using real-time traffic data.
- Connecting the system with **IoT-based traffic signals**.

15. Final Conclusion

The proposed system uses **Q-Learning** to control traffic signals dynamically. The RL agent observes traffic density, selects an appropriate signal action, receives a reward based on traffic reduction, and updates its Q-table. Through repeated learning, the agent can make better signal-control decisions and reduce vehicle waiting time.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	25	Demonstrates deep understanding of concepts with complete clarity	Explains concepts correctly with minor gaps	Partial understanding with errors or omissions	Unable to explain basic concepts
Application	25	Effectively applies concepts to new situations; solves problems logically	Applies concepts with minor errors	Limited ability to apply concepts; requires guidance	Unable to apply concepts effectively
Analytical Thinking	20	Critically analyzes scenarios with strong justification and reasoning	Provides reasonable analysis with some justification	Superficial analysis with weak reasoning	No analytical reasoning demonstrated
Communication	15	Communicates ideas clearly, confidently, and with appropriate terminology	Communicates clearly but with minor hesitation	Communication lacks clarity or structure	Unable to communicate ideas effectively
Response to Questions	15	Responds accurately and confidently, extending answers with depth	Responds correctly but with limited depth	Responds partially; requires prompting	Unable to respond to questions effectively